Topic 1. Problem set — λ-calculus

Use of AI assistants. Problems 1.1 to 1.4 are No AI. Problem 1.5 marks **each of its parts separately**, under the rules printed with it. At a defense you may be asked to redo any derivation from this sheet on paper, whichever problem it comes from.

Problem 1.1 (α-equivalence). No AI Twelve terms:

1. $\lambda x.\, \lambda y.\, x\ (y\ x)$
2. $\lambda a.\, \lambda b.\, a\ (b\ a)$
3. $\lambda y.\, \lambda x.\, y\ (x\ y)$
4. $\lambda p.\, \lambda q.\, q\ (p\ q)$
5. $\lambda x.\, \lambda y.\, y\ (x\ y)$
6. $\lambda u.\, \lambda v.\, v\ (u\ v)$
7. $\lambda x.\, \lambda y.\, x\ (x\ y)$
8. $\lambda m.\, \lambda n.\, m\ (m\ n)$
9. $\lambda x.\, \lambda y.\, x\ (y\ z)$
10. $\lambda a.\, \lambda b.\, a\ (b\ z)$
11. $\lambda a.\, \lambda b.\, a\ (b\ w)$
12. $\lambda z.\, \lambda y.\, z\ (y\ z)$

(a) Group them into α-equivalence classes.

(b) Three of the terms above mention z. Two of those three are α-equivalent to each other and the third is not. Say which, and what makes the difference.

Problem 1.2 (Call by name versus call by value). No AI Reduce the term

$$(\lambda x.\, \lambda y.\, y)\ \big((\lambda f.\, f\ (f\ z))\ (\lambda w.\, w)\big)\ c$$

as far as it goes, twice: once under **call by value** and once under **call by name**. Write one β-reduction per line and underline the redex you contract.

Then say how many steps each took, and what the longer one spent them on.

Problem 1.3 (Church pairs). No AI Recall the encodings from the lab:

$$\mathsf{tru} = \lambda t.\, \lambda f.\, t \quad \mathsf{fls} = \lambda t.\, \lambda f.\, f \quad \mathsf{pair} = \lambda f.\, \lambda s.\, \lambda b.\, b\ f\ s \quad \mathsf{fst} = \lambda p.\, p\ \mathsf{tru} \quad \mathsf{snd} = \lambda p.\, p\ \mathsf{fls}$$

(a) Write a term swap for which $\mathsf{swap}\ (\mathsf{pair}\ x\ y)$ reduces to $\mathsf{pair}\ y\ x$. Here you may use the aliases.

(b) Write the term that extracts y from $\mathsf{pair}\ x\ (\mathsf{pair}\ y\ z)$.

(c) Verify your answer to (b) by reduction, expanding every alias, so that no pair, fst, snd, tru or fls is left.

Problem 1.4 (Testing for zero). No AI A Church numeral applies its first argument to its second n times: $\mathsf{c}_n = \lambda s.\, \lambda z.\, s^n\ z$, where $s^0\ z$ is z, $s^1\ z$ is $s\ z$, $s^2\ z$ is $s\ (s\ z)$, and so on. The booleans are tru and fls as above.

(a) Write a term iszro such that $\mathsf{iszro}\ \mathsf{c}_0$ reduces to tru, and $\mathsf{iszro}\ \mathsf{c}_n$ reduces to fls for every $n > 0$. You may **not** use aliases: write the term out.

(b) Verify it on c_0 and on c_2 by full β-reduction, expanding every numeral and boolean. If a written-out term repeats, you may name it, provided you say what the name stands for.

(c) In one sentence, why does it work for *every* $n > 0$ and not only for the one you checked?

Problem 1.5 (Inlining, audited). A retry helper. It reports how many attempts fit in the time allowed.

```
timeout = 30                        # seconds we are willing to spend in total

def make_retry(limit):
    def retry(timeout):             # seconds to allow for one attempt
        spent = 0
        attempts = 0
        while spent < limit:
            attempts += 1
            spent += timeout
        return attempts
    return retry

run = make_retry(timeout)
print(run(10))
```

It prints 3: three attempts of ten seconds fit inside thirty. *Inlining* `make_retry` means replacing `make_retry(timeout)` by its body, with the parameter substituted.

(a) AI allowed Ask an assistant to do that inlining, and record what it produces. Any follow-up you put to it belongs here too, and in the transcript. Paste the program above, and then a prompt along these lines:

> Inline the call to `make_retry` in this program: replace `make_retry(timeout)` with the body of `make_retry`, substituting its parameter for the argument. The behaviour must not change.

Use a **basic, free** model, not the strongest you can reach: a strong one may just get it right.

(b) No AI Run its answer. If the program no longer prints 3, say what went wrong, in the language of the substitution rule. If it prints 3, read what it changed and say how that avoided the trap. Either way the evidence is the code it wrote, **not** its account of what it did.

(c) No AI Give a correct inlining.

(d) No AI Write the corresponding λ-term, of the shape $(\lambda limit.\, \lambda timeout.\, \ldots)\ timeout$, substitute correctly, and name the step the naive inlining skipped.

Use of AI assistants. Part (a) only. Alongside your solution, submit `transcript.md`: the whole exchange, your prompts and the assistant's replies, verbatim and unedited. Parts (b), (c) and (d) are answered in the solution file itself, in your own words, like every other problem on this sheet. You may **not** submit the assistant's answer as your own, and an answer to (b) saying *it was correct* with no check shown scores nothing. If you use no assistant, do the inlining yourself in (a) and answer (b) about your own first attempt.